

Your diagnosis is plausible: if the FP8 → BF16 staging checkpoint is numerically wrong, GPTQ/RTN/AWQ will only quantize corrupted tensors. For this model, I would **not** continue from bf16_temp. Start from a checkpoint that can already generate sane text in float/mixed precision, then run AWQ directly.

Mistral-Small-4-119B-2603 is a **119B MoE** model with **128 experts, 4 active experts**, and about **6.5B active parameters/token**, so MoE calibration is the main risk area. LLM Compressor specifically warns that MoE models need calibration-friendly MoE handling for GPTQ/AWQ/AutoRound, otherwise experts may be calibrated incorrectly and can produce unstable outputs. ([Hugging Face](#))

AWQ W4A16 is the right target for A100-80: vLLM supports AWQ on Ampere GPUs, and W4A16 keeps activations in 16-bit while storing weights in 4-bit integer form, giving roughly 3.7× compression. ([vLLM](#))

1. Use a verified source checkpoint

Best order of preference:

1. **Official / verified BF16 source**, not your manual FP8-dequant output.
2. A known working mixed BF16/F8 source, but only if it loads and generates correctly before quantization.
3. Existing AWQ checkpoint for comparison only, for example cyankiwi/Mistral-Small-4-119B-2603-AWQ-4bit, which is tagged for vLLM / compressed-tensors. ([Hugging Face](#))

There is also an Axolotl BF16 conversion listed as axolotl-ai-co/Mistral-Small-4-119B-2603-BF16, which may be a better quantization base than manually dequantizing FP8 yourself. ([Hugging Face](#))

Before quantizing, run a tiny float sanity check. If this fails, stop.

```
from transformers import AutoProcessor, Mistral3ForConditionalGeneration
```

```
import torch
```

```
MODEL_ID = "axolotl-ai-co/Mistral-Small-4-119B-2603-BF16" # or another verified source
```

```
processor = AutoProcessor.from_pretrained(MODEL_ID, trust_remote_code=True)
```

```

model = Mistral3ForConditionalGeneration.from_pretrained(
    MODEL_ID,
    dtype=torch.bfloat16,
    device_map="auto",
    trust_remote_code=True,
)

messages = [
    {"role": "user", "content": "Write one short sentence about Paris."}
]

inputs = processor.apply_chat_template(
    messages,
    add_generation_prompt=True,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
).to(model.device)

with torch.no_grad():
    out = model.generate(**inputs, max_new_tokens=32, do_sample=False)

print(processor.decode(out[0], skip_special_tokens=False))

```

2. Install a compatible stack

Mistral's own model card recommends current vLLM and Transformers mainline for this model family. ([Hugging Face](#))

```
uv venv --python 3.12
```

```
source .venv/bin/activate
```

```
uv pip install -U torch --index-url https://download.pytorch.org/whl/cu124
```

```
uv pip install -U git+https://github.com/huggingface/transformers.git
```

```
uv pip install -U datasets accelerate safetensors huggingface_hub mistral_common
```

```
git clone https://github.com/vllm-project/llm-compressor.git
```

```
cd llm-compressor
```

```
uv pip install -e .
```

For a 119B model, use LLM Compressor's **sequential** pipeline and offload. LLM Compressor recommends the sequential pipeline for large models, and its docs show `auto_offload` for models that do not fit fully in VRAM. ([vLLM](#))

3. AWQ W4A16 quantization script

Save as `quantize_mistral_small4_awq.py`.

```
import os
```

```
import torch
```

```
from datasets import load_dataset
```

```
from transformers import AutoProcessor, Mistral3ForConditionalGeneration
```

```
from compressed_tensors.offload import load_offloaded_model
```

```
from llmcompressor import oneshot
```

```
from llmcompressor.modifiers.quantization import QuantizationModifier
```

try:

```
from llmcompressor.modifiers.transform.awq import AWQModifier, AWQMapping
```

except ImportError:

```
from llmcompressor.modifiers.awq import AWQModifier, AWQMapping
```

```
MODEL_ID = os.environ.get(
```

```
    "MODEL_ID",
```

```
    "axolotl-ai-co/Mistral-Small-4-119B-2603-BF16",
```

```
)
```

```
OUT_DIR = os.environ.get(
```

```
    "OUT_DIR",
```

```
    "./Mistral-Small-4-119B-2603-AWQ-W4A16",
```

```
)
```

```
OFFLOAD_DIR = os.environ.get("OFFLOAD_DIR", "./offload_mistral_small4_awq")
```

```
NUM_CALIBRATION_SAMPLES = int(os.environ.get("NUM_CALIBRATION_SAMPLES", "512"))
```

```
MAX_SEQUENCE_LENGTH = int(os.environ.get("MAX_SEQUENCE_LENGTH", "2048"))
```

```
def render_chat(processor, tokenizer, messages):
```

```
    """
```

```
    Use the model's chat template for calibration.
```

```
    Deployment-like calibration data matters a lot for INT4.
```

```
"""
```

```
template_obj = processor if hasattr(processor, "apply_chat_template") else tokenizer
```

```
try:
```

```
    return template_obj.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False,
        reasoning_effort="none",
    )
```

```
except TypeError:
```

```
    return template_obj.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False,
    )
```

```
print(f"Loading processor from {MODEL_ID}")
```

```
processor = AutoProcessor.from_pretrained(MODEL_ID, trust_remote_code=True)
```

```
tokenizer = getattr(processor, "tokenizer", processor)
```

```
if getattr(tokenizer, "pad_token", None) is None:
```

```
    tokenizer.pad_token = tokenizer.eos_token
```

```

print("Preparing calibration dataset")

raw_ds = load_dataset(
    "HuggingFaceH4/ultrachat_200k",
    split=f"train_sft[:{NUM_CALIBRATION_SAMPLES * 2}]",
)

raw_ds = raw_ds.shuffle(seed=42).select(range(NUM_CALIBRATION_SAMPLES))

def preprocess(example):
    text = render_chat(processor, tokenizer, example["messages"])
    return {"text": text}

text_ds = raw_ds.map(preprocess, remove_columns=raw_ds.column_names)

def tokenize(example):
    return tokenizer(
        example["text"],
        truncation=True,
        max_length=MAX_SEQUENCE_LENGTH,
        padding=False,
        add_special_tokens=False,
    )

```

```
calib_ds = text_ds.map(tokenize, remove_columns=text_ds.column_names)
```

```
print("Loading model with CPU/disk offload")
```

```
os.makedirs(OFFLOAD_DIR, exist_ok=True)
```

```
with load_offloaded_model():
```

```
    model = Mistral3ForConditionalGeneration.from_pretrained(
        MODEL_ID,
        dtype=torch.bfloat16,
        device_map="auto_offload",
        offload_folder=OFFLOAD_DIR,
        trust_remote_code=True,
        low_cpu_mem_usage=True,
    )
```

```
# AWQ smoothing mappings.
```

```
# These cover common Mistral/Mixtral-style names:
```

```
# attention: q/k/v/o projections
```

```
# MLP experts: gate/up/down or w1/w3/w2 naming
```

```
awq_mappings = [
```

```
    AWQMapping(
        "re:. *input_layernorm$",
        [
```

```
        "re:.*q_proj$",
        "re:.*k_proj$",
        "re:.*v_proj$",
    ],
),
AWQMapping(
    "re:.*v_proj$",
    [
        "re:.*o_proj$",
    ],
),
AWQMapping(
    "re:.*post_attention_layernorm$",
    [
        "re:.*gate_proj$",
        "re:.*up_proj$",
        "re:.*w1$",
        "re:.*w3$",
    ],
),
AWQMapping(
    "re:.*up_proj$",
    [
        "re:.*down_proj$",
    ],
),
```

```
AWQMapping(  
    "re:.w3$",  
    [  
        "re:.w2$",  
    ],  
),  
]
```

Do not quantize lm_head, embeddings, routers/gates, norms, or vision/projection parts.

Quantize expert feed-forward Linear layers, but not the MoE router itself.

```
ignore = [  
    "lm_head",  
    "re:.embed_tokens.*",  
    "re:.norm.*",  
    "re:.router.*",  
    "re:.block_sparse_moe\\.gate.*",  
    "re:.mlp\\.gate$",  
    "re:.vision.*",  
    "re:.vision_model.*",  
    "re:.image.*",  
    "re:.multi_modal_projector.*",  
    "re:.mm_projector.*",  
]
```

```
recipe = [  
    AWQModifier(mappings=awq_mappings),  
    QuantizationModifier(  
        targets=["Linear"],  
        scheme="W4A16_ASYM",  
        ignore=ignore,  
    ),  
]
```

```
print("Starting AWQ W4A16 quantization")
```

```
oneshot(  
    model=model,  
    processor=processor,  
    tokenizer=tokenizer,  
    dataset=calib_ds,  
    recipe=recipe,  
    num_calibration_samples=NUM_CALIBRATION_SAMPLES,  
    max_seq_length=MAX_SEQUENCE_LENGTH,  
    batch_size=1,  
    pipeline="sequential",  
    moe_calibrate_all_experts=True,  
    output_dir=OUT_DIR,  
)
```

```
print(f"Saved quantized model to: {OUT_DIR}")
```

Run it:

```
MODEL_ID=axolotl-ai-co/Mistral-Small-4-119B-2603-BF16 \
```

```
OUT_DIR=./Mistral-Small-4-119B-2603-AWQ-W4A16 \
```

```
NUM_CALIBRATION_SAMPLES=512 \
```

```
MAX_SEQUENCE_LENGTH=2048 \
```

```
python quantize_mistral_small4_awq.py
```

LLM Compressor's oneshot flow performs post-training quantization, runs calibration for AWQ/GPTQ-style methods, and saves compressed safetensors to the output directory. Its default calibration settings are 512 samples, batch size 1, and MoE expert calibration is enabled by default, but I would still pass `moe_calibrate_all_experts=True` explicitly for this model. ([vLLM](#))

4. Serve with vLLM on A100-80

Do **not** start with the full 256k context. Even with 4-bit weights, KV cache for long context can blow up memory. Start with 8k or 16k.

```
vllm serve ./Mistral-Small-4-119B-2603-AWQ-W4A16 \
```

```
--host 0.0.0.0 \
```

```
--port 8000 \
```

```
--max-model-len 8192 \
```

```
--gpu-memory-utilization 0.92 \
```

```
--max-num-batched-tokens 4096 \
```

```
--max-num-seqs 8 \
```

```
--attention-backend FLASH_ATTN_MLA \
```

```
--tool-call-parser mistral \
```

```
--enable-auto-tool-choice \
```

```
--reasoning-parser mistral
```

If vLLM does not auto-detect the compressed-tensors format, add:

```
--quantization compressed-tensors
```

The original model card's vLLM example uses tensor parallelism for serving; on a single A100-80, AWQ may fit for short-context serving, but not for high-throughput or very long-context serving. ([Hugging Face](#))

5. Important checks if output is still garbage

First, confirm the source model generates sane text **before** AWQ. If it does not, the issue is still the source checkpoint, not the quantizer.

Second, inspect the actual module names before trusting the regexes:

```
for name, module in model.named_modules():
```

```
    cls = module.__class__.__name__.lower()
```

```
    if "linear" in cls and (
```

```
        "expert" in name
```

```
        or "moe" in name
```

```
        or "gate" in name
```

```
        or "router" in name
```

```
        or "proj" in name
```

```
    ):
```

```
        print(name)
```

Then adjust ignore and awq_mappings to match the real names. The router should stay unquantized; expert FFN weights can be quantized.

Third, use calibration prompts that look like your real serving prompts. vLLM's INT4 quantization guide recommends deployment-like calibration data, chat templates, roughly 512 samples, and sequence length around 2048. ([vLLM](#))

Fourth, avoid GPTQ for the first successful run. GPTQ can require huge Hessian memory, while AWQ is usually easier to get working for a very large MoE model. ([vLLM](#))

My strongest recommendation: **do not quantize from your manual FP8 → BF16 conversion.**

Get a float/mixed checkpoint that passes a basic generation test, then run the AWQ recipe above with MoE expert calibration enabled.